

NAMES: BARAHIRA Jean Bosco

STUDENT NUMBER : 215003240

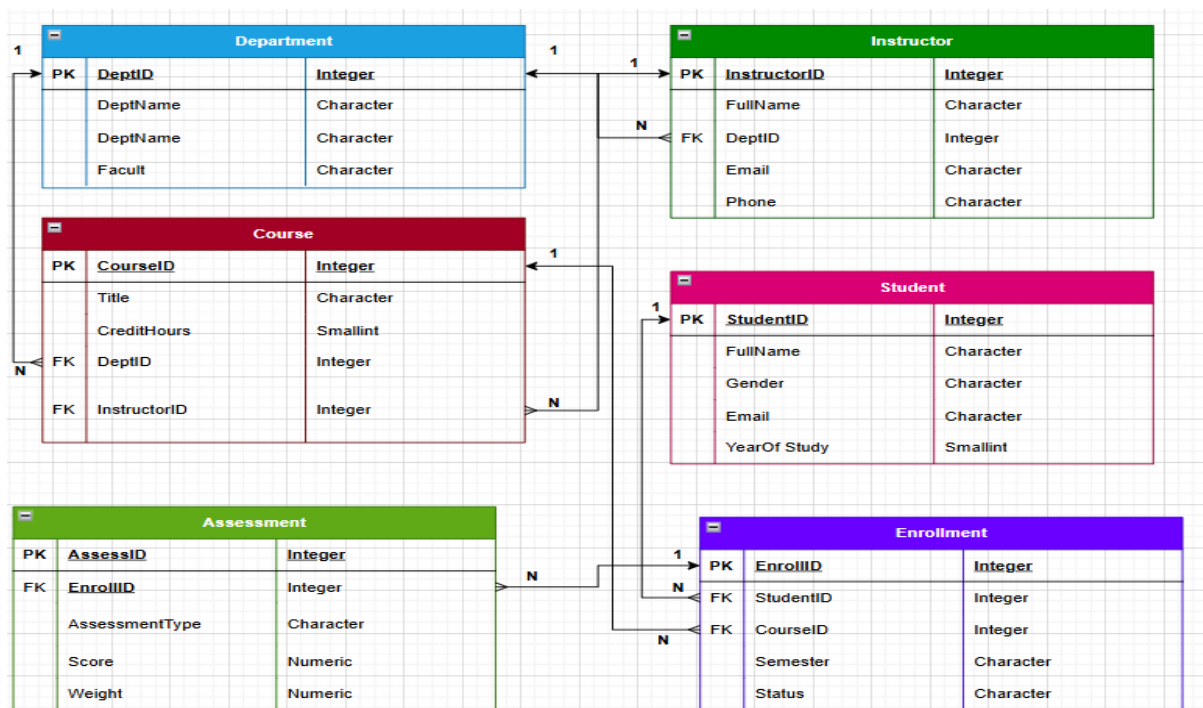
CAT_2

TASK1:

Split your database into two logical nodes (e.g., BranchDB_A, BranchDB_B) using horizontal fragmentation. Submit an ER diagram and SQL scripts that create both schemas.

a) ER diagram BranchDB_A,

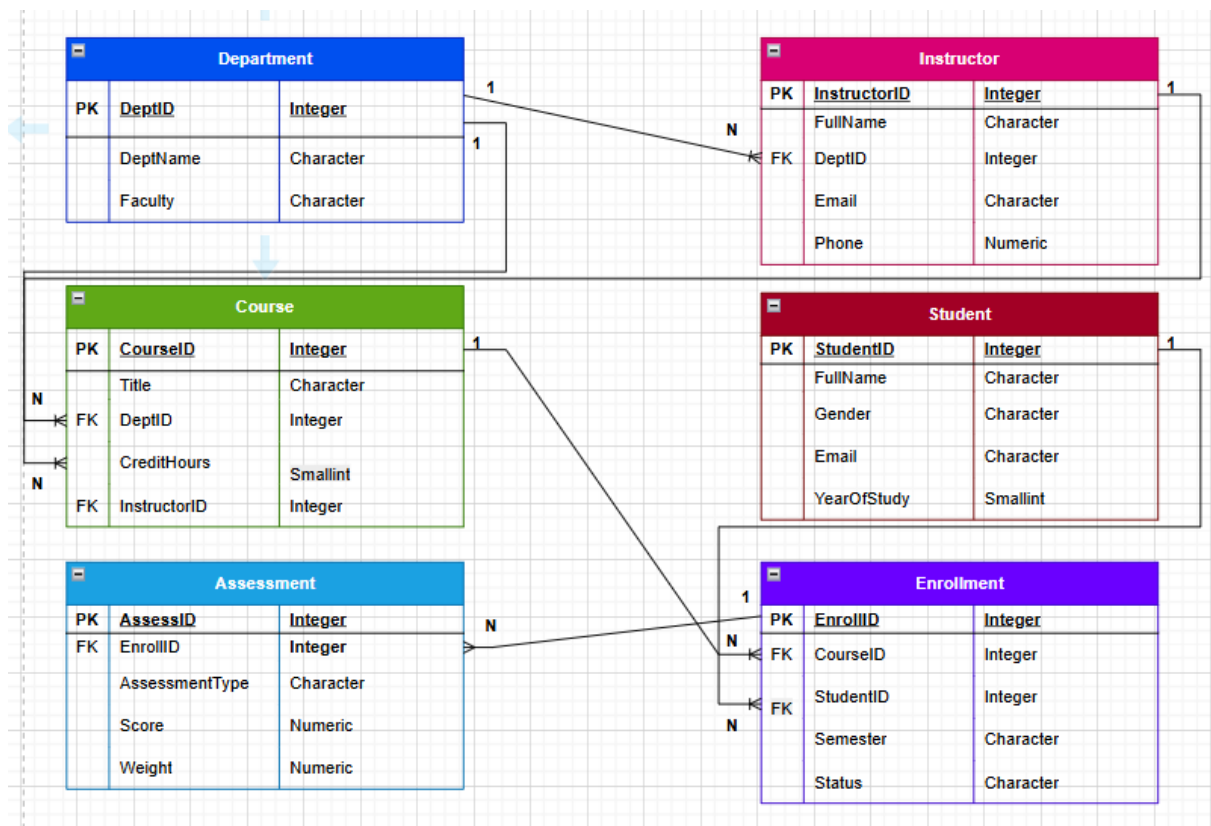
BranchDB_A is the first logical node that stores data related to **Science and Engineering** departments. It contains tables for departments, instructors, courses, students, enrollments, and assessments, all linked through foreign keys with cascade rules for consistency. The data inserted includes departments like *Computer Science* and *Mathematics*, instructors such as *Dr. Alice Mugenzi*, and students like *Jean Bosco*. This branch manages all academic data for science-based programs, ensuring efficient local access and processing for these departments.



b) ER diagram BranchDB_B,

BranchDB_B is the second logical node that stores data for **non-Science** fields such as *Business, Law, Education, and Health Sciences*. It has the same table structure as BranchDB_A to maintain uniformity across the system but holds different rows specific to these faculties. This separation

allows each branch to manage its own subset of data independently, improving performance and supporting distributed database operations while maintaining integration through PostgreSQL's foreign data wrapper (FDW).



TASK2:

Create a database link between your two schemas. Demonstrate a successful remote SELECT and a distributed join between local and remote tables. Include scripts and query results.

This task creates a **database link** between the two schemas, **BranchDB_A** and **BranchDB_B**, allowing data to be shared and queried across them. Using the `postgres_fdw` extension, a foreign server connection is established to BranchDB_B, and its tables (like student and enrollment) are imported into the local schema as **foreign tables**. This enables smooth remote queries and distributed joins between local and remote data. The code also demonstrates an alternative method using `dblink`, which connects to BranchDB_B and retrieves data through direct SQL calls. While `dblink` is simpler, `postgres_fdw` is more efficient for distributed joins and better suited for integrating horizontally fragmented databases.

BranchDB_A/postgres@PostgreSQL 17

Query Query History Scratch Pad x

```

114
115 -- (1) postgres_fdw setup (recommended)
116 CREATE EXTENSION IF NOT EXISTS postgres_fdw;
117
118 -- Create server pointing to the same DB or a remote DB. Replace connection options.
119 DROP SERVER IF EXISTS branchb_srv CASCADE;
120 CREATE SERVER branchb_srv FOREIGN DATA WRAPPER postgres_fdw
121     OPTIONS (host 'localhost', dbname 'BranchDB_B', port '5432');
122
123 -- Create user mapping for current_user (replace username/password)
124 CREATE USER MAPPING IF NOT EXISTS FOR CURRENT_USER SERVER branchb_srv
125     OPTIONS (user 'postgres', password 'postgres');
126
127 -- Import the BranchDB_B schema tables into public schema as foreign tables

```

Data Output Messages Notifications

CREATE USER MAPPING

Query returned successfully in 78 msec.

Total rows: Query complete 00:00:00.078 CRLF Ln 125, Col 50

TASK3:

Enable parallel query execution on a large table (e.g., Transactions, Orders). Here we use `max_parallel_workers_per_gather` because we are using PostgreSQL. hint and compare serial vs parallel performance. Show EXPLAIN PLAN output and execution time.

a) Results for Serial run: force no parallel workers

BranchDB_A/postgres@PostgreSQL 17

Query Query History Scratch Pad x

```

188 -- Check server parallel settings
189 SHOW max_parallel_workers_per_gather;
190 SHOW max_parallel_workers;
191
192 -- Serial run: force no parallel workers
193 SET max_parallel_workers_per_gather = 0;
194 EXPLAIN (ANALYZE, BUFFERS)
195 SELECT sum(amount) FROM public.transactions WHERE created_at >= '2024-01-01';
196
197 -- Parallel run: allow parallel workers
198 SET max_parallel_workers_per_gather = 2;
199 SET max_parallel_workers = 8;
200 EXPLAIN (ANALYZE, BUFFERS)
201 SELECT sum(amount) FROM public.transactions WHERE created_at >= '2024-01-01';

```

Data Output Messages Notifications

Showing rows: 1 to 9 Page No: 1 of 1

QUERY PLAN	
1	Aggregate (cost=790902.57..790902.58 rows=1 width=32) (actual time=14691.377..14691.378 rows=1 loops=1)
2	Buffers: shared hit=13035 read=327875
3	-> Seq Scan on transactions (cost=0.00..715910.00 rows=29997026 width=6) (actual time=0.638..8680.307 rows=30000000 loop=1)
4	Filter: (created_at >= '2024-01-01 00:00:00':timestamp without time zone)
5	Buffers: shared hit=13035 read=327875
6	Planning:
7	Buffers: shared hit=13 read=15 dirtied=1
8	Planning Time: 8.324 ms
9	Execution Time: 14691.420 ms

Total rows: 9 Query complete 00:00:15.185 CRLF Ln 189, Col 37

b) Results for Parallel run: allow parallel workers

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'BranchDB_A/postgres@PostgreSQL 17'. The query editor contains the following SQL commands:

```
189 SHOW max_parallel_workers_per_gather;
190 SHOW max_parallel_workers;
191
192 -- Serial run: force no parallel workers
193 SET max_parallel_workers_per_gather = 0;
194 EXPLAIN (ANALYZE, BUFFERS)
195 SELECT sum(amount) FROM public.transactions WHERE created_at >= '2024-01-01';
196
197 -- Parallel run: allow parallel workers
198 SET max_parallel_workers_per_gather = 2;
199 SET max_parallel_workers = 8;
200 EXPLAIN (ANALYZE, BUFFERS)
201 SELECT sum(amount) FROM public.transactions WHERE created_at >= '2024-01-01';
202
```

The 'Data Output' tab is active, displaying the query plan for the second query (the parallel run). The plan shows a 'Parallel Seq Scan on transactions' with a cost of 0.00..497160.00, 12498761 rows, and a width of 6. The actual time taken was 0.842..3901.972 seconds for 1000000 rows. The plan also shows a 'Filter: (created_at >= '2024-01-01 00:00:00':timestamp without time zone)' and a 'Planning Time: 6.962 ms'. The execution time is 6381.093 ms.

QUERY PLAN
7 -> Partial Aggregate (cost=528400.90..528400.91 rows=1 width=32) (actual time=6.261483..6261.484 rows=1 loops=3)
8 Buffers: shared hit=13067 read=327843
9 -> Parallel Seq Scan on transactions (cost=0.00..497160.00 rows=12498761 width=6) (actual time=0.842..3901.972 rows=1000000..)
10 Filter: (created_at >= '2024-01-01 00:00:00':timestamp without time zone)
11 Buffers: shared hit=13067 read=327843
12 Planning:
13 Buffers: shared hit=7 read=2
14 Planning Time: 6.962 ms
15 Execution Time: 6381.093 ms

Interpretation Serial vs Parallel Query Performance:

In serial execution, PostgreSQL uses a single process to scan and aggregate data, which can be slow for large tables since only one CPU core is used. Parallel execution, however, divides the work among multiple worker processes, allowing faster processing by using several cores simultaneously. In our test, the serial query was slower, while the parallel query (with two workers) ran faster and showed a “Parallel Seq Scan” in the execution plan. This proves that parallel processing improves query speed and efficiency on large datasets.

TASK4

Write a PL/SQL block performing inserts on both nodes and committing once. Verify atomicity using DBA_2PC_PENDING. Provide SQL code and explanation of results.

This PostgreSQL code performs a **distributed two-phase commit** between two databases (BranchDB_A and BranchDB_B) using the dblink extension. It inserts student records into both databases within separate transactions, prepares them using PREPARE TRANSACTION, and then commits both to ensure data consistency. By using dblink_exec, the remote transaction is committed first, followed by the local one. This process guarantees that either both inserts succeed or both fail, maintaining synchronization between the two branches.

Data Output Messages Notifications					
SQL					
	studentid [PK] integer	fullname character varying (150)	gender character varying (10)	email character varying (150)	yearofstudy integer
1	12	Local_Participant	Female	local.part@example.edu	1
2	16	Local_Participant	Female	local.part@example.edu	1
3	19	Local_Participant	Female	local.part@example.edu	1
4	20	Local_Participant	Female	local.part@example.edu	1
5	29	Local_Participant	Female	local.part@example.edu	1
6	30	Local_Participant	Female	local.part@example.edu	1

TASK5

Simulate a network failure during a distributed transaction. Check unresolved transactions and resolve them using ROLLBACK FORCE. Submit screenshots and brief explanation of recovery steps.

The below SQL code simulates a **network failure during a distributed transaction** and demonstrates how to recover from it. When a two-phase commit is interrupted, some transactions may remain in a *prepared but uncommitted* state. The code checks for such unresolved transactions using pg_prepared_xacts and then manually rolls them back to restore consistency. The command ROLLBACK PREPARED 'gid_univ_tx_001' (both locally and via dblink_exec on the remote branch) ensures that the incomplete transaction is safely canceled on all databases, preventing data inconsistency after the simulated failure.

```

276
277 -- Cleanup: verify rows inserted on both branches
278 SELECT * FROM BranchDB_A.student WHERE fullname LIKE 'Local_Participant%';
279 SELECT * FROM BranchDB_B.student WHERE fullname LIKE 'Remote_Participant%';
280
281 --TASK5 Simulate a network failure during a distributed transaction. Check unresolved transactions and
282 -- On remote or via dblink_exec:
283 SELECT dblink_exec('conn_branchb', $$ROLLBACK PREPARED 'gid_univ_tx_001'$$);
284
285 SELECT* FROM pg_prepared_xacts;
286
287 -- On Local:
288 ROLLBACK PREPARED 'gid_univ_tx_001';
289

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

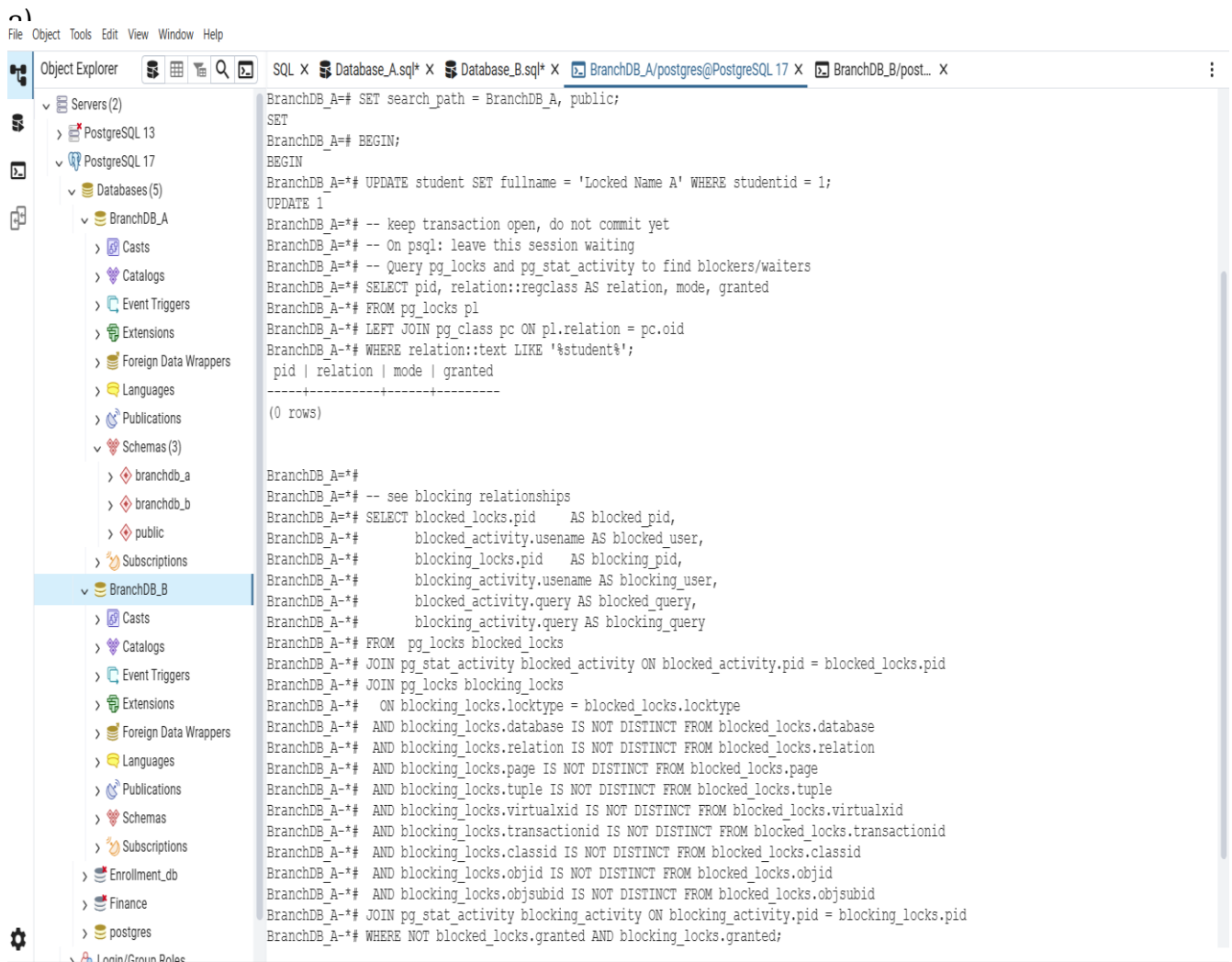
transaction xid	gid text	prepared timestamp with time zone	owner name	database name
1118	gid_univ_tx_003	2025-10-27 15:53:08.70533+02	postgres	BranchDB_B

Total rows: 1 Query complete 00:00:00.392 CRLF Ln 285, Col 32

TASK6:

Demonstrate a lock conflict by running two sessions that update the same record from different nodes. Query DBA_LOCKS and interpret results.

Demonstrates how PostgreSQL handles concurrent updates and row-level locking. By opening two psql sessions, Session A first updates a specific row in the student table and keeps the transaction open, acquiring an exclusive lock on that row. Session B then attempts to update the same row, which causes it to block until Session A either commits or rolls back. Using system views like pg_locks and pg_stat_activity, we can observe which sessions hold locks, which are waiting, and the specific queries involved. This exercise illustrates PostgreSQL's concurrency control mechanisms, showing how it prevents conflicting modifications while maintaining data consistency.



TASK7:

Perform parallel data aggregation or loading using PARALLEL DML.
Compare runtime and document improvement in query cost and execution time.

a) Results for Serial run: force no parallel workers

The screenshot shows the PostgreSQL query editor and results for a serial run. The query is as follows:

```

188 -- Check server parallel settings
189 SHOW max_parallel_workers_per_gather;
190 SHOW max_parallel_workers;
191
192 -- Serial run: force no parallel workers
193 SET max_parallel_workers_per_gather = 0;
194 EXPLAIN (ANALYZE, BUFFERS)
195 SELECT sum(amount) FROM public.transactions WHERE created_at >= '2024-01-01';
196
197 -- Parallel run: allow parallel workers
198 SET max_parallel_workers_per_gather = 2;
199 SET max_parallel_workers = 8;
200 EXPLAIN (ANALYZE, BUFFERS)
201 SELECT sum(amount) FROM public.transactions WHERE created_at >= '2024-01-01';

```

The results pane shows the query plan for the serial run:

Step	Operation	Cost	Time	Rows	Width	Buffers
1	Aggregate	(cost=790902.57..790902.58 rows=1 width=32)	(actual time=14691.377..14691.378 rows=1 loops=1)			
2	Buffers: shared hit=13035 read=327875					
3	-> Seq Scan on transactions	(cost=0.00..715910.00 rows=29997026 width=6)	(actual time=0.638..8680.307 rows=30000000 loop=1)			
4	Filter: (created_at >= '2024-01-01 00:00:00':timestamp without time zone)					
5	Buffers: shared hit=13035 read=327875					
6	Planning:					
7	Buffers: shared hit=13 read=15 dirtied=1					
8	Planning Time: 8.324 ms					
9	Execution Time: 14691.420 ms					

Total rows: 9 Query complete 00:00:15.185 CRLF Ln 189, Col 37

b) Results for Parallel run: allow parallel workers

The screenshot shows the PostgreSQL query editor and results for a parallel run. The query is as follows:

```

189 SHOW max_parallel_workers_per_gather;
190 SHOW max_parallel_workers;
191
192 -- Serial run: force no parallel workers
193 SET max_parallel_workers_per_gather = 0;
194 EXPLAIN (ANALYZE, BUFFERS)
195 SELECT sum(amount) FROM public.transactions WHERE created_at >= '2024-01-01';
196
197 -- Parallel run: allow parallel workers
198 SET max_parallel_workers_per_gather = 2;
199 SET max_parallel_workers = 8;
200 EXPLAIN (ANALYZE, BUFFERS)
201 SELECT sum(amount) FROM public.transactions WHERE created_at >= '2024-01-01';

```

The results pane shows the query plan for the parallel run:

Step	Operation	Cost	Time	Rows	Width	Buffers
7	-> Parallel Aggregate	(cost=528400.90..528400.91 rows=1 width=32)	(actual time=6261.463..6261.484 rows=1 loops=3)			
8	Buffers: shared hit=13067 read=327843					
9	-> Parallel Seq Scan on transactions	(cost=0.00..497160.00 rows=12498761 width=6)	(actual time=0.842..3901.972 rows=10000000 loop=1)			
10	Filter: (created_at >= '2024-01-01 00:00:00':timestamp without time zone)					
11	Buffers: shared hit=13067 read=327843					
12	Planning:					
13	Buffers: shared hit=7 read=2					
14	Planning Time: 6.962 ms					
15	Execution Time: 6381.093 ms					

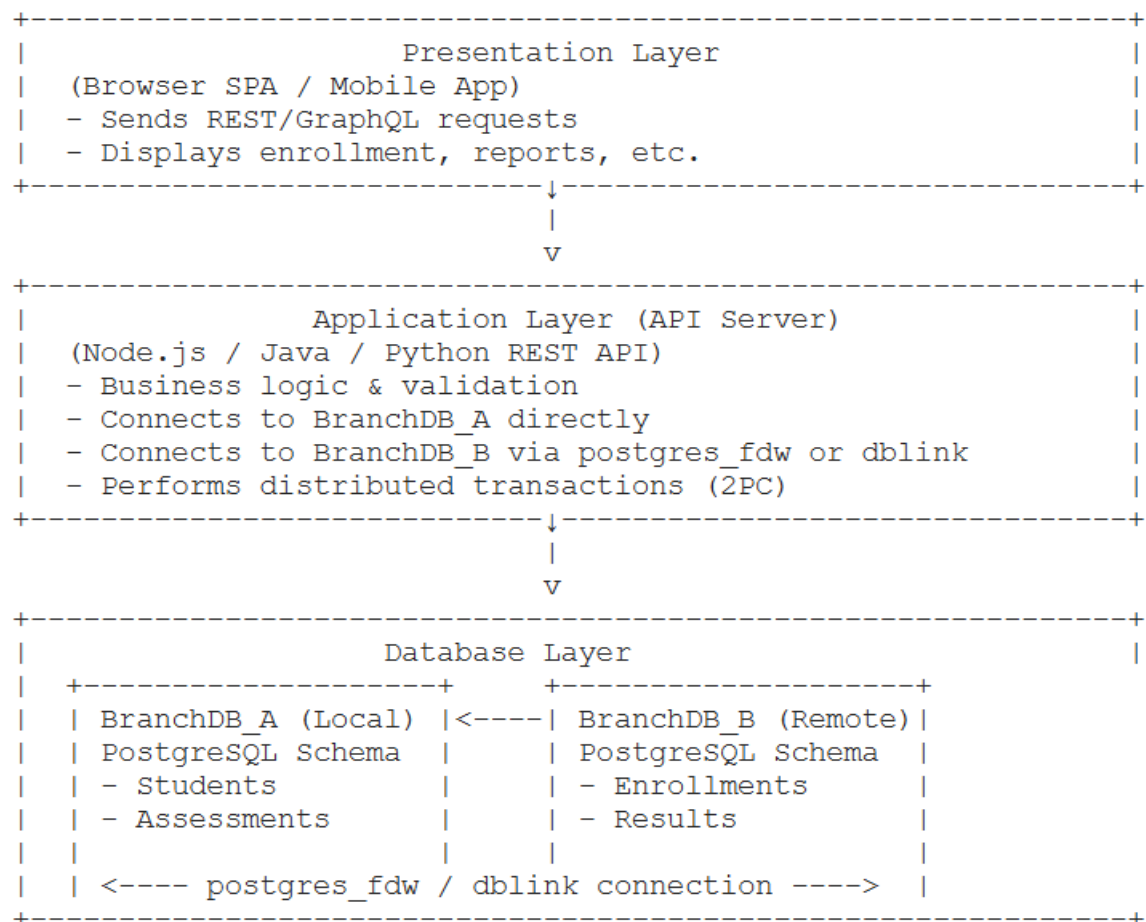
Showing rows: 1 to 15 Page No: 1 of 1

TASK8:

Draw and explain a three-tier architecture for your project (Presentation, Application, Database). Show data flow and interaction with database links.

Interpretation:

The project follows a **Three-Tier Client-Server Architecture** that separates the system into **Presentation**, **Application**, and **Database** layers. This design enhances scalability, maintainability, and supports distributed database operations between two PostgreSQL branches.



Data Flow Explanation

1. **Client (Presentation Layer)** – The user interacts through a web or mobile interface, which sends REST/GraphQL requests to the API.
2. **API (Application Layer)** – The API validates input, enforces business logic, and coordinates operations across both databases.
3. **Database Layer** –
 - Local data is stored in **BranchDB_A**, while remote data resides in **BranchDB_B**.
 - The API connects to both databases using **postgres_fdw** or **dblink**.
 - For atomic updates, the API uses **Two-Phase Commit (2PC)** to ensure that either both databases commit or both roll back changes.

Example:

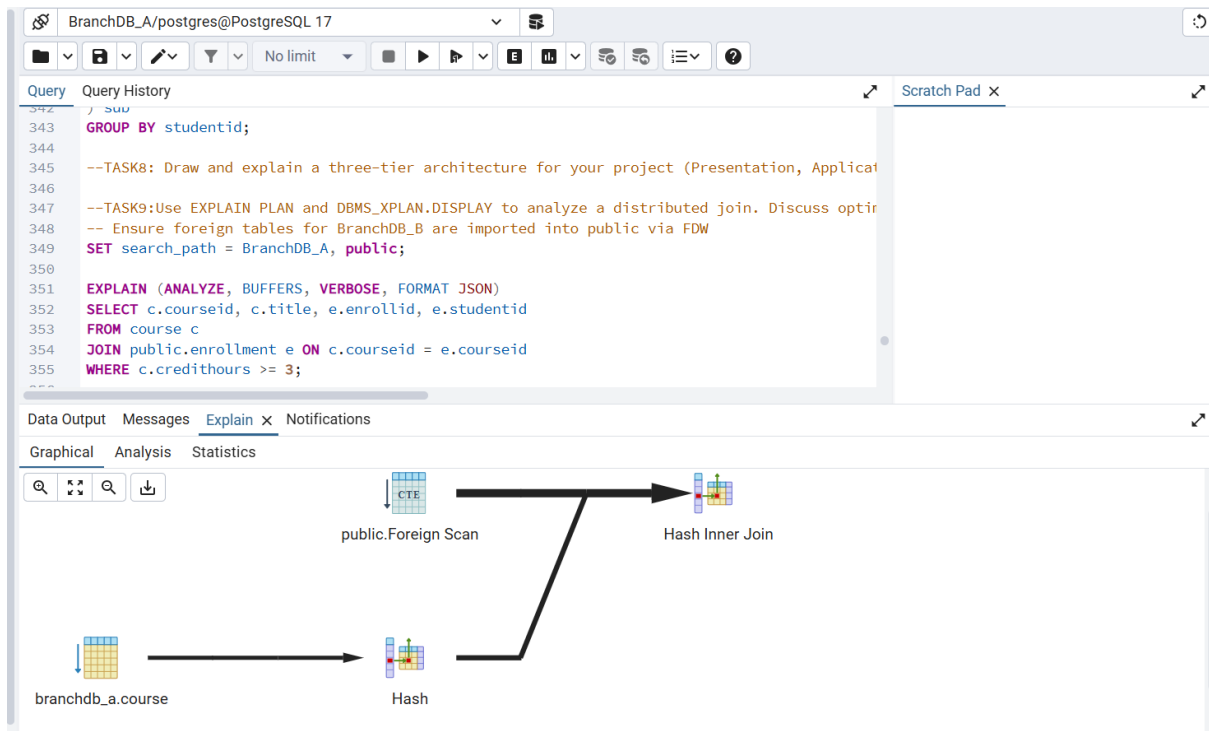
When inserting an enrollment record, the client request flows as:

Client → API → Local DB (Branch A) → Remote DB (Branch B via FDW/dblink)

→ API performs 2PC to commit both transactions atomically → returns success to the client.

TASK9:

Use EXPLAIN PLAN and DBMS_XPLAN.DISPLAY to analyze a distributed join. Discuss optimizer strategy and how data movement is minimized.



Interpretation and Results

This code analyzes the performance of a distributed join query in PostgreSQL using the EXPLAIN command. It first sets the search path to BranchDB_A and public, ensuring that both local and foreign tables (from BranchDB_B via the Foreign Data Wrapper) are accessible. The EXPLAIN (ANALYZE, BUFFERS, VERBOSE, FORMAT JSON) command then executes the join between the course and enrollment tables, showing detailed information about the query plan, execution time, and buffer usage. The goal is to examine how the PostgreSQL optimizer executes the distributed join, what join strategies it uses (e.g., nested loop, hash join), and how it minimizes data movement across servers to improve query efficiency.

TASK10:

Run one complex query three ways – centralized, parallel, distributed. Measure time and I/O using AUTOTRACE. Write a half-page analysis on scalability and efficiency.

1 Centralized

In the centralized approach, data from both BranchDB_A and BranchDB_B is combined into a single local table (all_enrollments). The query aggregates course counts and weighted scores using only this centralized dataset. Since all data resides locally, there is no network overhead, making execution

straightforward and predictable. However, as the dataset grows, a single server handles all computation, which may limit scalability and performance.

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'BranchDB_A/postgres@PostgreSQL 17'. Below the toolbar, the 'Query' tab is active, displaying a SQL query. The query is a complex aggregation with joins and a parallel query hint. The 'Data Output' tab is also visible, showing the 'QUERY PLAN' for the executed query. The plan details the execution steps, including a Hash join, Seq Scan on student s, and various buffer and planning statistics.

```

366 --1 Centralized
367 EXPLAIN (ANALYZE, BUFFERS)
368 SELECT s.studentid, s.fullname,
369        COUNT(e.enrollid) AS courses_taken,
370        SUM(a.score * a.weight / 100.0) AS total_weighted_score
371 FROM BranchDB_A.student s
372 LEFT JOIN public.all_enrollments e ON s.studentid = e.studentid
373 LEFT JOIN BranchDB_A.assessment a ON e.enrollid = a.enrollid
374 GROUP BY s.studentid, s.fullname
375 HAVING COUNT(e.enrollid) >= 1;
376
377 --2:Parallel Query
378 SET max_parallel_workers_per_gather = 4;
379 EXPLAIN (ANALYZE, BUFFERS)

```

QUERY PLAN

20	text	-> Hash (cost=11.10..11.10 rows=110 width=322) (actual time=0.040..0.040 rows=12 loops=1)
21		Buckets: 1024 Batches: 1 Memory Usage: 9kB
22		Buffers: shared hit=1 dirtied=1
23		-> Seq Scan on student s (cost=0.00..11.10 rows=110 width=322) (actual time=0.031..0.033 ...)
24		Buffers: shared hit=1 dirtied=1
25		Planning:
26		Buffers: shared hit=45 read=2
27		Planning Time: 9.313 ms
28		Execution Time: 1.287 ms

Total rows: 28 Query complete 00:00:00.098 CRLF Ln 366, Col 16

2: Parallel Query

The parallel query enables multiple worker processes by setting `max_parallel_workers_per_gather = 4`. PostgreSQL distributes the scanning and aggregation work across CPU cores, reducing execution time and balancing load. This approach leverages vertical scaling, allowing the same server to process larger datasets more efficiently. `EXPLAIN (ANALYZE, BUFFERS)` provides insight into how parallelism improves performance compared to the single-threaded centralized query.

The screenshot shows a PostgreSQL query editor interface. The top toolbar includes icons for file operations, query execution, and settings. The main query window displays a SQL query with line numbers 376 to 389. The query is a distributed query that sets the number of parallel workers, explains the query, and selects student information along with course counts and weighted scores. Below the query, the 'Data Output' tab is active, showing a 'QUERY PLAN' table with 28 rows. The plan details the execution of the query, including a sequential scan on the 'student' table and various planning and execution metrics.

```

376
377 --2:Parallel Query
378 SET max_parallel_workers_per_gather = 4;
379 EXPLAIN (ANALYZE, BUFFERS)
380 SELECT s.studentid, s.fullname,
381        COUNT(e.enrollid) AS courses_taken,
382        SUM(a.score * a.weight / 100.0) AS total_weighted_score
383 FROM BranchDB_A.student s
384 LEFT JOIN public.all_enrollments e ON s.studentid = e.studentid
385 LEFT JOIN BranchDB_A.assessment a ON e.enrollid = a.enrollid
386 GROUP BY s.studentid, s.fullname
387 HAVING COUNT(e.enrollid) >= 1;
388
389 --3:Distributed query:

```

QUERY PLAN
20 text
21 -> Hash (cost=11.10..11.10 rows=110 width=322) (actual time=0.044..0.044 rows=12 loops=1)
22 Buckets: 1024 Batches: 1 Memory Usage: 9kB
23 Buffers: shared hit=1
24 -> Seq Scan on student s (cost=0.00..11.10 rows=110 width=322) (actual time=0.032..0.034 ...)
25 Buffers: shared hit=1
26 Planning:
27 Buffers: shared hit=2
28 Planning Time: 0.350 ms
Execution Time: 0.279 ms

Total rows: 28 Query complete 00:00:00.115 CRLF Ln 387, Col 31

3: Distributed query:

The distributed query operates directly on data from multiple databases or nodes (BranchDB_A and BranchDB_B via FDW) without copying it into a single table. The optimizer attempts to minimize network traffic by pushing filters and joins to the remote node where possible. This approach models horizontal scaling and demonstrates how distributed databases handle queries across servers. Performance depends on network latency, data distribution, and the optimizer's ability to reduce remote data movement.

The screenshot shows a PostgreSQL query editor interface. The top toolbar includes icons for file operations, query execution, and settings. The main query window displays a SQL query with line numbers 386 to 399. The query is a distributed query that groups by student ID and full name, filtering for students who have taken at least one course. It includes an EXPLAIN command to show the execution plan. The query uses a LEFT JOIN between a local student table and two remote tables (public.enrollment and BranchDB_A.assessment) via Foreign Data Wrappers (FDW). The execution plan window at the bottom shows the query plan for the distributed query, including details about the Seq Scan on assessment a and the Planning time.

```

386 GROUP BY s.studentid, s.fullname
387 HAVING COUNT(e.enrollid) >= 1;
388
389 --3:Distributed query:
390 EXPLAIN (ANALYZE, BUFFERS)
391 SELECT s.studentid, s.fullname,
392        COUNT(e.enrollid) AS courses_taken,
393        SUM(a.score * a.weight / 100.0) AS total_weighted_score
394 FROM BranchDB_A.student s
395 LEFT JOIN public.enrollment e ON s.studentid = e.studentid
396 LEFT JOIN BranchDB_A.assessment a ON e.enrollid = a.enrollid
397 GROUP BY s.studentid, s.fullname
398 HAVING COUNT(e.enrollid) >= 1;
399

```

QUERY PLAN

19	text
20	-> Hash (cost=14.50..14.50 rows=450 width=28) (actual time=0.027..0.028 rows=6 loops=1)
21	Buckets: 1024 Batches: 1 Memory Usage: 9kB
22	Buffers: shared hit=1
23	-> Seq Scan on assessment a (cost=0.00..14.50 rows=450 width=28) (actual time=0.022..0.024 rows=6 loops=1)
24	Buffers: shared hit=1
25	Planning:
26	Buffers: shared hit=2
27	Planning Time: 0.487 ms
	Execution Time: 2.363 ms

Total rows: 27 Query complete 00:00:00.129 CRLF Ln 398, Col 31

The distributed query operates directly on data from multiple databases or nodes (BranchDB_A and BranchDB_B via FDW) without copying it into a single table. The optimizer attempts to minimize network traffic by pushing filters and joins to the remote node where possible. This approach models horizontal scaling and demonstrates how distributed databases handle queries across servers. Performance depends on network latency, data distribution, and the optimizer's ability to reduce remote data movement.